

¿Por qué mi celular cierra apps *de la nada*?

Gestión de memoria y el *OOM Killer*

Santiago Arath Bello Sánchez
Facultad de Ingeniería
Universidad Nacional Autónoma de México
Ciudad de México, México

David Baruc López Romero
Facultad de Ingeniería
Universidad Nacional Autónoma de México
Ciudad de México, México

Resumen—La gestión de memoria principal es uno de los pilares críticos en el diseño de los sistemas operativos modernos. Tradicionalmente, en las arquitecturas de escritorio, el agotamiento de la memoria física se mitiga mediante técnicas de paginación y espacio de intercambio (swap). En este contexto, la terminación forzada de procesos a través del Out Of Memory (OOM) Killer del kernel de Linux se considera un último recurso ante una solicitud de memoria no manejable por el sistema mediante la administración de recursos disponibles. Sin embargo, la adopción masiva de dispositivos móviles, caracterizados por recursos de hardware estrictamente limitados y con restricciones energéticas, forzó un cambio de paradigma radical. Este documento analiza la evolución y divergencia en las políticas de gestión de memoria, contrastando el OOM Killer tradicional con el Low Memory Killer Daemon (lmkd) implementado en Android. Se examina cómo, en los sistemas móviles, la finalización de procesos en segundo plano ha dejado de ser un estado de error para convertirse en el flujo normal de operación, exigiendo que las aplicaciones sean diseñadas para preservar su estado y terminar 'con gracia'. Finalmente, se desmitifican prácticas comunes de los usuarios frente a la heurística del sistema operativo para la retención en caché.

Index Terms—Sistemas Operativos, Gestión de Memoria, OOM Killer, LMKD, Android, Kernel de Linux, Paginación.

I. INTRODUCCIÓN

La multitarea en los dispositivos móviles modernos presenta una ilusión de fluidez y persistencia ininterrumpida. Todo usuario de un teléfono inteligente ha experimentado el siguiente escenario: interactuar con una aplicación que demanda altos recursos de hardware, cambiar a una aplicación de mensajería para responder un texto rápido y, al regresar a la tarea original, descubrir que esta se ha reiniciado desde cero, perdiendo el progreso no guardado.

Este fenómeno, comúnmente percibido por el usuario como un error o un fallo de la aplicación, es en realidad el resultado de una decisión crítica del sistema operativo frente a un problema fundamental de arquitectura: la memoria física es finita. Cuando la memoria RAM se agota, el sistema debe elegir a qué procesos terminar para evitar que el dispositivo colapse por completo. El presente trabajo investiga la evolución de la gestión de memoria, analizando cómo el agotamiento de recursos pasó de ser tratado como una falla crítica en sistemas de escritorio, a convertirse en el flujo de operación normal en entornos móviles.

II. EL PARADIGMA TRADICIONAL (SISTEMAS DE ESCRITORIO)

En la arquitectura tradicional del Kernel de Linux, la terminación forzada de procesos se encuentra diseñada como un mecanismo de supervivencia extrema y no como el flujo de control habitual. El Out Of Memory (OOM) Killer por defecto se encuentra enlazado al asignador de memoria y opera de manera estricta como un último recurso ya que, su ejecución se posterga de manera deliberada hasta que tanto la memoria física como las particiones de espacio de intercambio o swap se encuentran agotadas casi en su totalidad, ya que la heurística del sistema tiene como principal objetivo minimizar la finalización de procesos [1].

Si bien esta política conservadora protege el proceso y estado de todas las aplicaciones, lo termina haciendo a expensas del rendimiento global. Al evitar la terminación de un proceso que consume demasiada memoria, el sistema incurre en una sobrecarga inasumible provocada por el intercambio constante de páginas de memoria hacia el disco físico (thrashing). Para un entorno de escritorio esto se traduce en una degradación severa donde el sistema operativo pierde temporalmente su capacidad de respuesta antes de que el OOM Killer finalmente intervenga.

III. EL CAMBIO DE PARADIGMA (DISPOSITIVOS MÓVILES)

En los comienzos de los sistemas operativos móviles, las estrictas limitaciones de los recursos y la ausencia de un espacio de intercambio físico (swap) tradicional fueron pauta de un gran reto para la ingeniería. Históricamente, las plataformas móviles tuvieron que adoptar soluciones como ZRAM en lugar de particiones en disco debido a la limitada vida útil y capacidad de los medios de almacenamiento basados en memoria flash NAND [2].

A diferencia de un entorno de escritorio, en un dispositivo móvil el número de aplicaciones en memoria tiende a incrementar constantemente con el tiempo debido al modo de uso por parte del usuario, y la memoria física no puede ser ampliada por el usuario para mitigar esa escasez. Debido a que las aplicaciones en segundo plano llegan a consumir espacio de memoria total, el sistema operativo se ve forzado a gestionar de manera agresiva estos recursos para mantener los servicios base de la plataforma. Esto provocó que la política de gestión

se trasladara hacia el espacio de usuario (userland) mediante 'demonios' especializados [2], [3].

IV. OOM KILLER VS LMKD (LOW MEMORY KILLER DAEMON EN ANDROID)

Para lidiar con las restricciones de hardware, Android implementa una jerarquía de reclamación de memoria más escalonada que el Kernel estándar de Linux, activando sus defensas en el siguiente orden: ZRAM, Swap, LMKD y como última instancia, OOM Killer [3].

Cuando la utilización de la memoria alcanza niveles peligrosos en gestión para el correcto funcionamiento del dispositivo, es decir, niveles cercanos al 80 % o 90 %, el sistema no puede depender únicamente del intercambio de páginas y activa entonces el Low Memory Killer Daemon (LMKD) y el OOM Killer para reclamar así grandes bloques de memoria de forma inmediata. El LMKD actúa eliminando procesos de usuario en función de su puntuación (OOM score) una vez que la memoria disponible cae por debajo de los umbrales estrictamente predefinidos.

El sistema clasifica agresivamente a los procesos para proteger la experiencia interactiva; por ejemplo, las aplicaciones en segundo plano reciben una puntuación (OOM score) alta (como 15), y son las primeras víctimas, mientras que las aplicaciones en primer plano mantienen un valor de 0, y los procesos nativos del sistema reciben un valor muy bajo, o incluso negativo (como -16) para así evitar su terminación. Estas operaciones de 'asesinato' de procesos resultan tolerables para la experiencia del usuario final debido a que la mayoría de aplicaciones móviles operan bajo un modelo cliente-servidor, lo cual les permite restaurar su estado interno rápidamente al reconectarse a sus servidores remotos una vez que el usuario reinicia su ejecución [3].

Para ilustrar el impacto de esta heurística, resulta fundamental observar el contraste de hardware en el ecosistema actual. Un equipo de gama alta maneja los umbrales del LMKD de manera radicalmente distinta a un teléfono de gama de entrada. En el dispositivo de mayor capacidad, el kernel de Android permite un nivel de *overcommit* sumamente agresivo, albergando decenas de procesos cacheados en segundo plano bajo la premisa de que rara vez todos exigirán su cuota máxima de memoria de forma simultánea. Por el contrario, en un equipo con recursos limitados, el LMKD actúa de manera predictiva y estricta, sacrificando procesos en segundo plano constantemente para garantizar que la aplicación en primer plano disponga de los recursos mínimos necesarios para mantener la fluidez de la interfaz gráfica.

V. EL DISEÑO DE LAS APLICACIONES (MURIENDO CON GRACIA)

El sistema operativo Android, procura mantener los procesos de las aplicaciones en ejecución el mayor tiempo posible. Sin embargo, ante el agotamiento constante de recursos, el sistema se ve obligado a eliminar procesos recurrentemente para así recuperar memoria y dar espacio a nuevas tareas [4]. [4]. Para minimizar el impacto en la experiencia del

usuario, Android termina los procesos en función de su nivel de importancia. Bajo este esquema heurístico, los procesos en primer plano (foreground), que son aquellos con los que el usuario interactúa directamente en el instante actual, reciben la máxima prioridad y están protegidos contra el OOM Killer [4].

Para los procesos en segundo plano (background), en lugar de generar fallos abruptos (crashes), la arquitectura móvil exige que las aplicaciones operen de manera cooperativa. A medida que la memoria RAM se llena, el sistema notifica proactivamente a las aplicaciones mediante métodos de devolución de llamada (callbacks) como *onTrimMemory()* [5]. Esta señal actúa como una advertencia temprana, permitiendo a la aplicación liberar recursos no críticos (como imágenes en caché o conexiones de red inactivas) antes de convertirse en un objetivo para el Low Memory Killer Daemon (LMKD) [5].

Si la memoria sigue siendo insuficiente y la terminación del proceso es inminente, Android invoca el método *onSaveInstanceState()* [6]. Con este mecanismo obliga a la aplicación a serializar y guardar el estado actual de su interfaz de usuario en un paquete de datos ligero llamado Bundle. Cuando el usuario decida regresar a la aplicación desde la vista de 'recientes', el sistema operativo no inicializa el proceso desde cero, en su lugar, le entrega a la aplicación el estado previamente guardado, forzándola a reconstruir esa interfaz exactamente como el usuario la dejó. Como resultado de este diseño, la muerte del proceso se vuelve un evento ligero e invisible, creando la ilusión para el usuario de que la aplicación siempre se mantuvo en la memoria principal [6].

VI. EL MITO DE CERRAR APLICACIONES MANUALMENTE

Una creencia popular entre los usuarios de Smartphones es que cerrar manualmente las aplicaciones desde la vista de tareas recientes (forzando así su terminación) ahorra energía y mejora el rendimiento del dispositivo. Sin embargo, desde la perspectiva de la arquitectura de los sistemas operativos móviles, esta práctica resulta contraproducente. Históricamente, la demanda de rendimiento de los smartphones incrementa a un ritmo mucho más acelerado que las mejoras tecnológicas en su capacidad de baterías, lo cual genera un conflicto directo entre la ejecución de tareas pesadas y la autonomía del dispositivo [7].

La memoria RAM por su naturaleza de hardware consume una cantidad de energía constante independientemente de si se encuentra vacía o llena de datos. Al mantener una aplicación suspendida en segundo plano, el sistema operativo permite algo conocido como 'arranque en caliente' (Hot Start). En este estado, la aplicación se reanuda de manera casi instantánea, ya que sus estructuras de datos, subprocesos y recursos de interfaz gráfica ya se encuentran mapeados en memoria.

Por el contrario, cuando un usuario cierra una aplicación manualmente, obliga al sistema a eliminarla de la memoria. La próxima vez que se solicite, el sistema operativo deberá ejecutar un 'arranque en frío' (Cold Start). Inicializar una aplicación desde cero conlleva una penalización masiva de rendimiento debido a las operaciones intensivas de I/O en el

almacenamiento físico y a los picos abruptos del procesador. Específicamente, el sistema debe invertir ciclos de la CPU críticos en tareas computacionales pesadas, como la resolución de símbolos de objetos compartidos (SO-Symbol Lookup), y la decodificación de jerarquías visuales complejas desde archivos XML (XML-UI conversion) [3].

Los dispositivos móviles enfrentan recursos estrictamente limitados en cuanto a capacidad de batería, almacenamiento y rendimiento del procesador [7]. En consecuencia, el acto de cerrar aplicaciones repetidamente para 'liberar memoria' genera un mayor desgaste energético en el hardware y anula los beneficios heurísticos del sistema, disminuyendo la vida útil de la batería al forzar arranques en frío que exigen un alto costo computacional [3], [7].

VII. CONCLUSIONES

Tal como se ha analizado, la gestión de la memoria expone una profunda divergencia arquitectónica que se ha consolidado en las últimas dos décadas. En los sistemas de escritorio tradicionales, el cierre forzado de una aplicación en ejecución sigue siendo tratado por el sistema como un evento extremo y una medida de último recurso.

Sin embargo, el paradigma de los dispositivos móviles tomó un camino completamente distinto desde sus cimientos. A casi 20 años de la adopción masiva de este modelo, se ha normalizado e interiorizado que cualquier aplicación que no se encuentre ejecutándose en el primer plano pueda ser finalizada como parte del flujo normal de operación. Este cambio obligó a la industria del software a diseñar aplicaciones resilientes, capaces de manejar estas interrupciones con gracia restaurar su estado sin pérdida de información. Resulta fascinante observar cómo este eficiente modelo móvil, basado en la finalización constante e invisible de tareas, representa un cambio de mentalidad que, hasta el día de hoy, no ha logrado permear en las interfaces tradicionales de cómputo.

REFERENCIAS

- [1] W. Chen, A. Pi, S. Wang, and X. Zhou, "Os-augmented oversubscription of opportunistic memory with a user-assisted oom killer," in *Proceedings of the 20th International Middleware Conference*, 2019, pp. 28–40.
- [2] B. Zolnierkiewicz, "Efficient memory management on mobile devices," *Published on: Sep*, vol. 17, 2013.
- [3] G. Lim, D. Kang, M. Ham, and Y. I. Eom, "Swam: Revisiting swap and oomk for improving application responsiveness on mobile devices," in *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*, 2023, pp. 1–15.
- [4] K. Khanmohammadi, M. R. Rejali, and A. Hamou-Lhadj, "Understanding the service life cycle of android apps: An exploratory study," in *Proceedings of the 5th Annual ACM CCS Workshop on Security and Privacy in Smartphones and Mobile Devices*, 2015, pp. 81–86.
- [5] Android Developers, "Descripción general de la administración de memoria," <https://developer.android.com/topic/performance/memory?hl=es-419>, accedido: 1 de mayo de 2026.
- [6] —, "Guardar el estado de la iu," <https://developer.android.com/guide/fragments/saving-state?hl=es-419>, accedido: 1 de mayo de 2026.
- [7] H. Qian and D. Andresen, "Jade: Reducing energy consumption of android app," *International Journal of Networked and Distributed Computing*, vol. 3, no. 3, pp. 150–158, 2015.